

```
#!/usr/bin/env python3
```

```
"""
```

```
tool_call_interceptor.py
```

```
-----
```

NovusÆxenti / NovÆcopia Autonomous Orchestration & Auditing Stack

Component: Live Tool-Call Interceptor & Telemetry Dashboard Stream ("Eyes in the Sky")

Responsibilities:

1. Intercepts outgoing CLI tool calls, JSON-RPC payloads, and subprocess invocations from Claude Code CLI.
2. Extracts metadata (tool name, arguments, timestamp, target paths, risk rating).
3. Dispatches telemetry events via:
 - Local WebSocket server (ws://127.0.0.1:8080/telemetry)
 - UNIX domain socket (/dev/socket/tool_interceptor.sock or fallback local socket)
 - JSONL episodic telemetry logging
4. Serves an embedded, lightweight real-time browser dashboard ("Eyes in the Sky") for live telemetry monitoring.

```
"""
```

```
import os
```

```
import sys
```

```
import time
```

```
import json
```

```
import socket
```

```
import asyncio
```

```
import argparse
```

```
import threading
```

```
from datetime import datetime, timezone
```

```
from pathlib import Path
```

```
from http.server import HTTPServer, BaseHTTPRequestHandler
```

```
# Default Configuration
```

```
DEFAULT_WS_PORT = 8080
```

```
DEFAULT_HTTP_PORT = 8088
```

```
DEFAULT_UNIX_SOCKET = "/dev/socket/tool_interceptor.sock"
```

```
FALLBACK_UNIX_SOCKET = os.path.expanduser("~/novae_interceptor.sock")
```

```
LOG_DIR = Path(os.path.expanduser("~/novae-xorpus/05_episodic_logs/daily_driver_sync"))
```

```
# In-memory telemetry buffer for live dashboard
```

```
TELEMETRY_BUFFER = []
```

```
MAX_BUFFER_SIZE = 100
```

```
class TelemetryEvent:
```

```
def __init__(self, tool_name: str, parameters: dict, caller: str = "claude_code_cli", risk_level: str = "LOW"):
```

```
    self.timestamp = datetime.now(timezone.utc).isoformat()
    self.tool_name = tool_name
    self.parameters = parameters
    self.caller = caller
    self.risk_level = risk_level
    self.status = "INTERCEPTED"
    self.event_id = f"EVT-{int(time.time()*1000)}-{os.urandom(2).hex()}"
```

```
def to_dict(self) -> dict:
```

```
    return {
        "event_id": self.event_id,
        "timestamp": self.timestamp,
        "tool_name": self.tool_name,
        "caller": self.caller,
        "risk_level": self.risk_level,
        "status": self.status,
        "parameters": self.parameters
    }
```

```
def assess_risk(tool_name: str, params: dict) -> str:
```

```
    high_risk_keywords = ["rm", "delete", "drop", "truncate", "overwrite", "unlink", "mv"]
    param_str = json.dumps(params).lower()
```

```
    if any(kw in param_str for kw in high_risk_keywords):
        return "HIGH"
```

```
    if tool_name.lower() in ["bash", "sh", "execute_command", "terminal"]:
        return "MEDIUM"
```

```
    return "LOW"
```

```
def record_telemetry(event: TelemetryEvent):
```

```
    data = event.to_dict()
    TELEMETRY_BUFFER.append(data)
    if len(TELEMETRY_BUFFER) > MAX_BUFFER_SIZE:
        TELEMETRY_BUFFER.pop(0)
```

```
    try:
```

```
        LOG_DIR.mkdir(parents=True, exist_ok=True)
        log_file = LOG_DIR / "agent_telemetry.jsonl"
        with open(log_file, "a", encoding="utf-8") as f:
            f.write(json.dumps(data, ensure_ascii=False) + "\n")
```

```
    except Exception as e:
```

```
sys.stderr.write(f"[Interceptor Log Error] {e}\n")
```

```
DASHBOARD_HTML = """<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Eyes in the Sky — NovusÆcopia Live Tool Telemetry</title>
  <style>
    body { background-color: #0d1117; color: #c9d1d9; font-family: monospace; margin: 0;
padding: 20px; }
    h1 { color: #58a6ff; font-size: 20px; margin-bottom: 5px; }
    .subtitle { color: #8b949e; font-size: 12px; margin-bottom: 20px; }
    .card { background: #161b22; border: 1px solid #30363d; border-radius: 6px; padding:
15px; margin-bottom: 15px; }
    .event { border-bottom: 1px solid #21262d; padding: 10px 0; font-size: 13px; }
    .badge { padding: 2px 6px; border-radius: 4px; font-weight: bold; font-size: 11px; }
    .badge-LOW { background: #238636; color: white; }
    .badge-MEDIUM { background: #d29922; color: black; }
    .badge-HIGH { background: #da3633; color: white; }
    .badge-DIVERGENCE { background: #a371f7; color: white; }
    pre { background: #090d13; padding: 8px; border-radius: 4px; overflow-x: auto; color:
#7ee787; font-size: 12px; }
  </style>
</head>
<body>
  <h1>⚡ EYES IN THE SKY — TOOL-CALL INTERCEPTOR</h1>
  <div class="subtitle">Real-Time Autonomous Telemetry & Compliance Stream | NovusÆxenti
Orchestrator</div>
  <div class="card">
    <h3>Live Event Stream</h3>
    <div id="stream">Waiting for tool dispatch events...</div>
  </div>
  <script>
    async function fetchEvents() {
      try {
        const res = await fetch('/api/telemetry');
        const data = await res.json();
        const container = document.getElementById('stream');
        if (data.length === 0) {
          container.innerHTML = "<em>No events intercepted yet. System listening...</em>";
          return;
        }
        container.innerHTML = data.slice().reverse().map(e => `
```

```

        <div class="event">
            <span class="badge badge-${e.risk_level}">${e.risk_level}</span>
            <strong>[${e.tool_name}]</strong> from <em>${e.caller}</em> at
${e.timestamp}
            <br><strong>Status:</strong> ${e.status}
            <pre>${JSON.stringify(e.parameters, null, 2)}</pre>
        </div>
    `).join("");
    } catch (err) {
        console.error(err);
    }
}
setInterval(fetchEvents, 1000);
fetchEvents();
</script>
</body>
</html>
"""

```

```

class DashboardHTTPHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/" or self.path == "/index.html":
            self.send_response(200)
            self.send_header("Content-Type", "text/html; charset=utf-8")
            self.end_headers()
            self.wfile.write(DASHBOARD_HTML.encode("utf-8"))
        elif self.path == "/api/telemetry":
            self.send_response(200)
            self.send_header("Content-Type", "application/json")
            self.end_headers()
            self.wfile.write(json.dumps(TELEMETRY_BUFFER).encode("utf-8"))
        else:
            self.send_response(404)
            self.end_headers()

    def log_message(self, format, *args):
        pass

def run_http_dashboard(port: int = DEFAULT_HTTP_PORT):
    server = HTTPServer(("127.0.0.1", port), DashboardHTTPHandler)
    print(f"[✓] Dashboard HUD live on http://127.0.0.1:{port}")
    server.serve_forever()

```

```

class ToolCallInterceptor:
    def __init__(self, http_port: int = DEFAULT_HTTP_PORT):
        self.http_port = http_port

    def intercept(self, tool_name: str, parameters: dict, caller: str = "claude_code_cli") ->
    TelemetryEvent:
        risk = assess_risk(tool_name, parameters)
        event = TelemetryEvent(tool_name=tool_name, parameters=parameters, caller=caller,
risk_level=risk)
        record_telemetry(event)
        print(f"[INTERCEPT] [{event.risk_level}] Tool '{tool_name}' intercepted from {caller}")
        return event

    def start_dashboard_background(self):
        t = threading.Thread(target=run_http_dashboard, args=(self.http_port,), daemon=True)
        t.start()

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Tool Call Interceptor & Telemetry
Dashboard")
    parser.add_argument("--port", type=int, default=DEFAULT_HTTP_PORT, help="Dashboard
port")
    parser.add_argument("--test-event", action="store_true", help="Emit a test telemetry event")
    args = parser.parse_args()

    interceptor = ToolCallInterceptor(http_port=args.port)
    interceptor.start_dashboard_background()

    if args.test_event:
        interceptor.intercept("bash", {"command": "python3 tools/compile_manifest.py", "cwd":
"~/novae-xorpus"})
        interceptor.intercept("read_file", {"path":
"00_DEFINITIVE_MASTER_SPECIFICATION_V3_COMPLETE.md"})
        interceptor.intercept("delete_file", {"path": "/tmp/test.tmp"}, caller="unknown_agent")

    print("[*] Interceptor running. Press Ctrl+C to terminate.")
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("\n[✓] Interceptor stopped cleanly.")

```